Problems with Forward/Backward Variable Selection

Horacio Gomez-Acevedo

Variable selection problems

Even though it is a very common practice to reduce the number of covariates using step-wise selection, it is in general not a good practice.

The following simulations are based on Dr. Harrell's notes https://hbiostat.org/rmsc/multivar

The setup is to create 8 independent random variables $x_1, \dots, x_8$, and a simple linear regression model for the independent variable l_p defined in terms of a given subset of variables

$$l_p = x_1 + x_2 + 0.5x_3 + 0.4x_7 + \varepsilon \text{ where } \varepsilon = N(0, \sigma^2)$$

The standard deviation is set to 2.5 by default but it can be changed by the user. The model in R is written as

$$y \sim x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 + x_8$$

Note that we omit the coefficients in the formula and we are also considering the y-intercept β_0 . Also, the Akaike's information criteria is used to carry on with variable reduction.

```
require(MASS)    # provides stepAIC function
```

```
Loading required package: MASS
```

```
require(leaps)   # provides regsubsets function
```

```
Loading required package: leaps
```

```
sim <- function(n, sigma=2.5, method=c('stepaic', 'leaps'),
                pr=FALSE, prcor=FALSE, dataonly=FALSE) {
  method <- match.arg(method)
  if(uncorrelated) {
    x1 <- rnorm(n)
    x2 <- rnorm(n)
    x3 <- rnorm(n)
    x4 <- rnorm(n)
    x5 <- rnorm(n)
    x6 <- rnorm(n)
    x7 <- rnorm(n)
    x8 <- rnorm(n)
    }
  else {
      x1 <- rnorm(n)
      x2 <- x1 + 2.0 * rnorm(n)       # was + 0.5 * rnorm(n)
      x3 <- rnorm(n)
      x4 <- x3 + 1.5 * rnorm(n)
      x5 <- x1 + rnorm(n)/1.3
      x6 <- x2 + 2.25 * rnorm(n)      # was rnorm(n)/1.3
      x7 <- x3 + x4 + 2.5 * rnorm(n)  # was + rnorm(n)
      x8 <- x7 + 4.0 * rnorm(n)       # was + 0.5 * rnorm(n)
  }
  z <- cbind(x1,x2,x3,x4,x5,x6,x7,x8)
  if(prcor) return(round(cor(z), 2))
  lp <- x1 + x2 + .5*x3 + .4*x7
  y <- lp + sigma*rnorm(n)
  if(dataonly) return(list(x=z, y=y))
  if(method == 'leaps') {
    s     <- summary(regsubsets(z, y))
    best  <- which.max(s$adjr2)
    xvars <- s$which[best, -1]   # remove intercept
    ssr   <- s$rss[best]
    p     <- sum(xvars)
    xs    <- if(p == 0) 'none' else paste((1 : 8)[xvars], collapse='')
    if(pr) print(xs)
    ssesw <- (n - 1) * var(y) - ssr
    s2s   <- ssesw / (n - p - 1)
    yhat  <- if(p == 0) mean(y) else fitted(lm(y ~ z[, xvars]))
  }
  f <- lm(y ~ x1 + x2 + x3 + x4 + x5 + x6 + x7 + x8)
  if(method == 'stepaic') {
```

```
    g <- stepAIC(f, trace=0)
    p <- g$rank - 1
    xs <- if(p == 0) 'none' else
      gsub('[ <br>+x]','', as.character(formula(g))[3])
    if(pr) print(formula(g), showEnv=FALSE)
    ssesw <- sum(resid(g)^2)
    s2s   <- ssesw/g$df.residual
    yhat  <- fitted(g)
  }
  # Set SSEsw / (n - gdf - 1) = true sigma^2
  gdf <- n - 1 - ssesw / (sigma^2)
  # Compute root mean squared error against true linear predictor
  rmse.full <- sqrt(mean((fitted(f) - lp) ^ 2))
  rmse.step <- sqrt(mean((yhat - lp) ^ 2))
  list(stats=c(n=n, vratio=s2s/(sigma^2),
         gdf=gdf, apparentdf=p, rmse.full=rmse.full, rmse.step=rmse.step),
       xselected=xs)
}

rsim <- function(B, n, method=c('stepaic', 'leaps')) {
  method <- match.arg(method)
  xs <- character(B)
  r <- matrix(NA, nrow=B, ncol=6)
  for(i in 1:B) {
    w     <- sim(n, method=method)
    r[i,] <- w$stats
    xs[i] <- w$xselected
  }
  colnames(r) <- names(w$stats)
  s <- apply(r, 2, median)
  p <- r[, 'apparentdf']
  s['apparentdf'] <- mean(p)
  print(round(s, 2))
  print(table(p))
  cat('Prob[correct model]=', round(sum(xs == '1237')/B, 2), '\n')
}
```

Then, we will run some simulations

```
uncorrelated <- TRUE
sim(50000, prcor=TRUE)
```

```
      x1    x2    x3    x4    x5   x6    x7    x8
x1  1.00  0.00  0.01  0.00 -0.01 0.00  0.01  0.00
x2  0.00  1.00  0.00  0.00 -0.01 0.00  0.00  0.00
x3  0.01  0.00  1.00  0.00 -0.01 0.00  0.00  0.00
x4  0.00  0.00  0.00  1.00  0.00 0.00 -0.01  0.01
x5 -0.01 -0.01 -0.01  0.00  1.00 0.00 -0.01  0.00
x6  0.00  0.00  0.00  0.00  0.00 1.00  0.01  0.00
x7  0.01  0.00  0.00 -0.01 -0.01 0.01  1.00 -0.01
x8  0.00  0.00  0.00  0.01  0.00 0.00 -0.01  1.00
```

```
set.seed(11)
m <- 'leaps'              # all possible regressions stopping on R2adj
rsim(100, 20, method=m)  # actual model found twice out of 100
```

```
         n     vratio        gdf apparentdf  rmse.full  rmse.step
     20.00       0.94       5.32       4.10       1.62       1.58
p
 1  2  3  4  5  6  7  8
 3 14 18 22 27 11  4  1
Prob[correct model]= 0.02
```

Then, what happens when n increases

```
rsim(100, 200, method=m)
```

```
         n     vratio        gdf apparentdf  rmse.full  rmse.step
    200.00       0.43     115.59       5.09       0.52       0.51
p
 3  4  5  6  7
 3 25 43 18 11
Prob[correct model]= 0.18
```

One more, with a large vector size

```
rsim(100,2000,method=m)
```

```
         n     vratio        gdf apparentdf  rmse.full  rmse.step
   2000.00       0.39    1228.32       5.32       0.16       0.16
p
 4  5  6  7
19 42 27 12
Prob[correct model]= 0.19
```

What if we increase the number of simulations?

```
rsim(1000,200,method=m)
```

```
         n     vratio        gdf apparentdf  rmse.full  rmse.step
    200.00       0.43     116.41       5.13       0.51       0.50
p
  2   3   4   5   6   7   8
  1  33 233 406 236  80  11
Prob[correct model]= 0.18
```

Conclusion

Even when you consider that you have a *good model* it is necessary to run some simulations to see if your model is stable.